

TESTs - Lesson 17 - Gas optimization and Smart Contract security

What is the primary reason why blockchains like Ethereum have gas limits for transactions?

- A To encourage developers to write more complex code.
- B To prevent Denial of Service (DoS) attacks by limiting the computational work per transaction.**
- C To generate higher revenue for miners.
- D To restrict the number of users on the network.

What is the main problem with using an "unbounded loop" (e.g., for every user, do something...) in a Solidity smart contract, particularly concerning gas limits?

- A It makes the code harder to read and maintain.
- B It can consume an unpredictable amount of gas, potentially exceeding the block limit and causing transaction failure or DoS.**
- C It leads to infinite loops that crash the entire blockchain.
- D It makes smart contracts less decentralized.

What is the historical trend of the Ethereum average gas limit?

- A It has consistently decreased over time due to network upgrades.
- B It has remained fixed at 30 million units since Ethereum's inception.
- C It stayed around 30 million units for a long time before being periodically increased.**
- D It fluctuates randomly with no clear trend.

The sortProposals function in the SortedBallot.sol contract uses a while(true) loop to implement a bubble sort algorithm. What makes it problematic for on-chain use?

- A It is highly efficient for small data sets.
- B It has an unpredictable number of iterations, making its gas consumption unbounded.**
- C It requires manual intervention to break the loop.
- D It only works with predefined data types.

Explain the concept of "Amortization of Work" in the context of smart contract development.

- ☒ A Dividing a single, large computational task into smaller, sequential operations executed across multiple transactions to stay within gas limits.
- ☐ B Minimizing the number of state variable updates to reduce gas costs.
- ☐ C Outsourcing all smart contract logic to off-chain servers.
- ☐ D Consolidating multiple small transactions into one large transaction.

How can developers mitigate the risk of unbounded loops leading to transaction failures or Denial of Service (DoS) attacks in smart contracts?

- ☒ A By performing computations off-chain whenever possible and using granular, iterative on-chain operations with state variables to track progress.
- ☐ B By increasing the block gas limit for individual transactions.
- ☐ C By implementing more complex sorting algorithms.
- ☐ D By deploying contracts during off-peak network hours.

Why is sorting data directly on the blockchain using inefficient algorithms like bubble sort generally discouraged?

- ☒ A These algorithms are computationally expensive and can easily hit the block gas limit, rendering the contract unusable.
- ☐ B Sorting algorithms introduce randomness, which is not secure for smart contracts.
- ☐ C On-chain sorting is too slow for real-time applications.
- ☐ D It compromises the decentralization of the network.

What is a key difference between memory variables and state variables in Solidity concerning gas consumption and persistence across transactions?

- ☐ A Memory variables are more expensive but persist across transactions, while state variables are cheaper but temporary.
- ☒ B State variables are expensive and persistent across transactions, whereas memory variables are cheaper and temporary (lost after function execution).
- ☐ C There is no significant difference in gas consumption between them.
- ☐ D Memory variables are stored on the blockchain, and state variables are stored off-chain.

What is the core problem that gas limits address in the EVM?

- ☒ A To prevent network congestion and resource exhaustion from malicious or inefficient operations.
- ☐ B To simplify smart contract development.
- ☐ C To ensure all transactions are processed instantly.
- ☐ D To reduce the cost of deploying contracts.